

20250614改进训练

由于ResNet16不能够展现出合格的性能发挥，现更改为ResNet50进行训练。

同时优化图像处理，取消全高斯噪音：

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(0.1, 0.1, 0.1),
    transforms.RandomAffine(degrees=0, translate=(0.05, 0.05), scale=(0.95,
1.05)),
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.5, scale=(0.01, 0.1), ratio=(0.3, 3.3)),
])
```

优化权重计算方式，改为动态计算：

```
from sklearn.utils.class_weight import compute_class_weight
targets = [label for _, label in train_dataset]
weights = compute_class_weight('balanced', classes=[0, 1], y=targets)
class_weights = torch.tensor(weights, dtype=torch.float32).to(device)
```

修改损失函数为FocalLoss：

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

其中， p_t 是预测为真实类别的概率， α_t 为类别权重， γ 为调制因子（控制关注难类别的程度，通常为0.2）

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class FocalLoss(nn.Module):
    def __init__(self, alpha=None, gamma=2.0, reduction='mean'):
        """
        alpha: 类别权重（例如 [0.25, 0.75]），为 None 则默认每类 1.0
        gamma: 调制因子，控制“难例”的强化程度
        reduction: 'mean', 'sum', or 'none'
        """
```

```

super(FocalLoss, self).__init__()
self.alpha = alpha
self.gamma = gamma
self.reduction = reduction

if isinstance(alpha, (list, torch.Tensor)):
    self.alpha = torch.tensor(alpha, dtype=torch.float32)
else:
    self.alpha = None

def forward(self, inputs, targets):
    # inputs: [B, C], raw logits
    # targets: [B], class indices
    ce_loss = F.cross_entropy(inputs, targets, reduction='none')

    pt = torch.exp(-ce_loss) # pt = softmax prob of the true class
    focal_term = (1 - pt) ** self.gamma

    if self.alpha is not None:
        alpha_factor = self.alpha[targets].to(inputs.device)
        focal_term = alpha_factor * focal_term

    loss = focal_term * ce_loss

    if self.reduction == 'mean':
        return loss.mean()
    elif self.reduction == 'sum':
        return loss.sum()
    return loss

```

考虑使用余弦退火：

```
scheduler = CosineAnnealingLR(optimizer, T_max=100, eta_min=1e-6)
```

这将使学习率以余弦函数下降，最小值为 1e-6

在原测试集上：

	precision	recall	f1-score	support
0	0.9677	0.7353	0.8357	204
1	0.9393	0.9940	0.9659	840
accuracy			0.9435	1044
macro avg	0.9535	0.8647	0.9008	1044

weighted avg	0.9448	0.9435	0.9404	1044
--------------	--------	--------	--------	------

经过干扰后：

	precision	recall	f1-score	support
0	0.8750	0.7549	0.8105	204
1	0.9424	0.9738	0.9578	840
accuracy			0.9310	1044
macro avg	0.9087	0.8644	0.8842	1044
weighted avg	0.9292	0.9310	0.9291	1044

说明该模型抗干扰能力优越，然而在0类别上仍然欠缺。继续训练。

模型分割策略有问题，导致模型只能读取一个标签。现修正如下：

加载整个数据集并获取标签：

```
dataset = ImageFolder(root='train', transform=your_transform)
targets = [label for _, label in dataset]
```

划分索引：

```
train_idx, val_idx = train_test_split(
    list(range(len(dataset))),
    test_size=0.2,
    stratify=targets,
    random_state=42
)
```

为指定不同的变换，自定义Subset类：

```
class CustomSubset(torch.utils.data.Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform
```

```
def __getitem__(self, index):
    x, y = self.subset[index]
    if self.transform:
        x = self.transform(x)
    return x, y

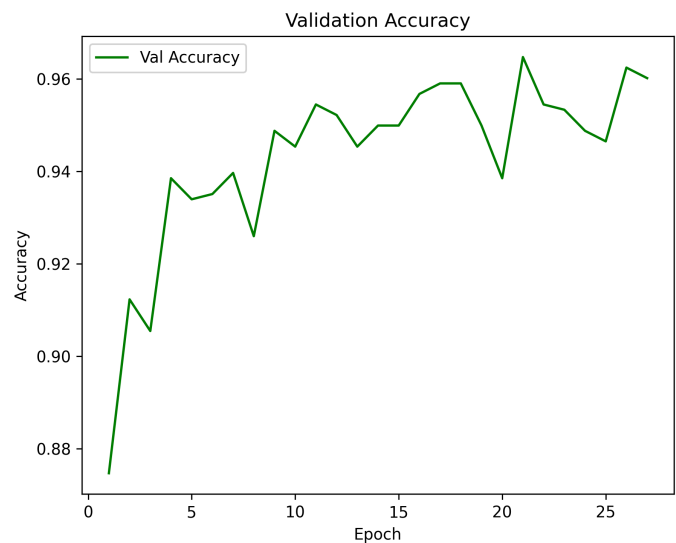
def __len__(self):
    return len(self.subset)
```

包装成数据集：

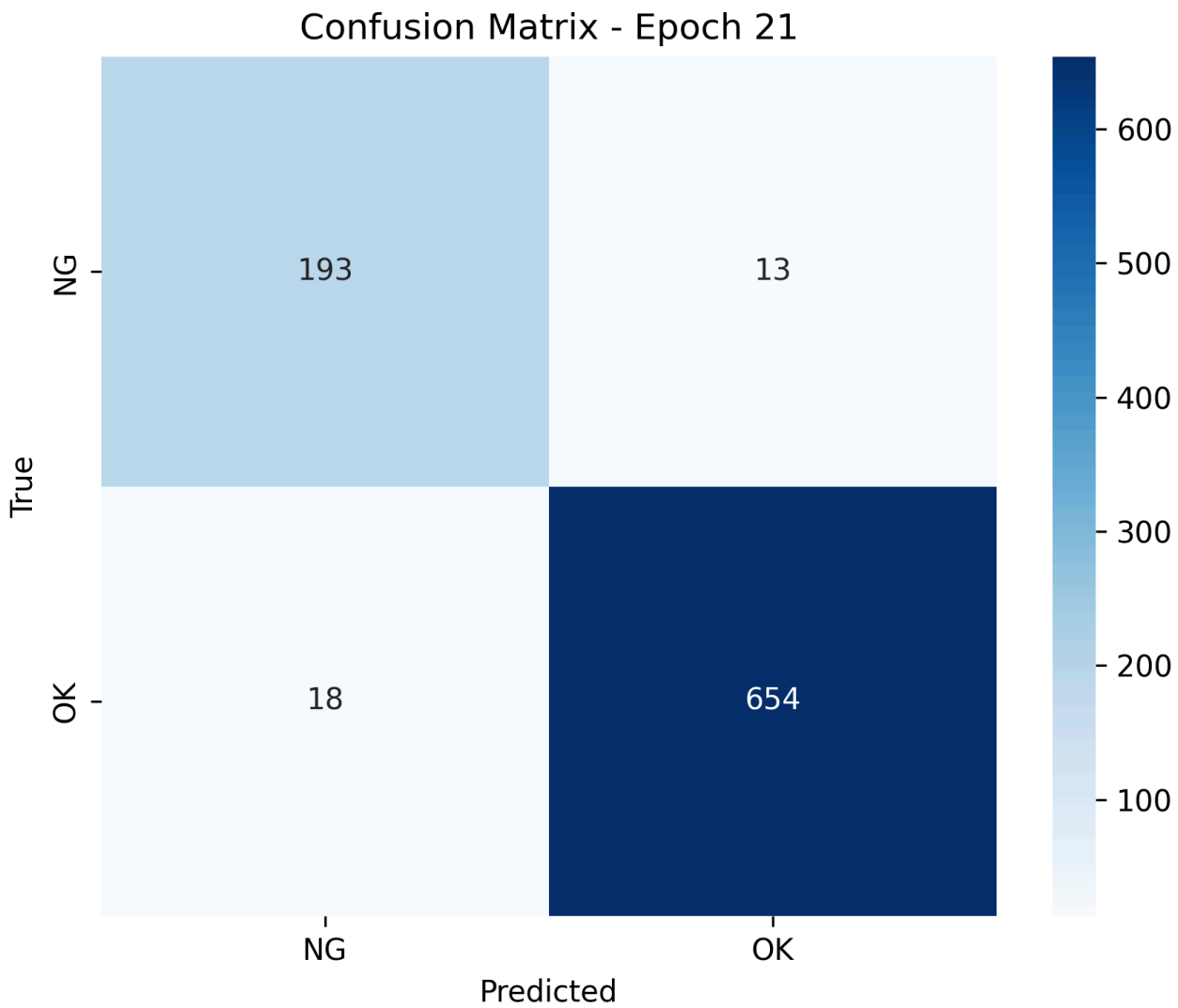
```
train_dataset = CustomSubset(SubsetData, train_idx),
transform=train_transform)
val_dataset = CustomSubset(SubsetData, val_idx),
transform=val_transform)
```

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
```

在第一次训练中，25轮后触发早停：



准确率在20轮左右出现峰值：



在测试集上进行测试：

	precision	recall	f1-score	support
0	0.8851	0.7549	0.8148	204
1	0.9425	0.9762	0.9591	840
accuracy			0.9330	1044
macro avg	0.9138	0.8655	0.8869	1044
weighted avg	0.9313	0.9330	0.9309	1044